



Saad Mughal

F2023-009

Total Marks: 100

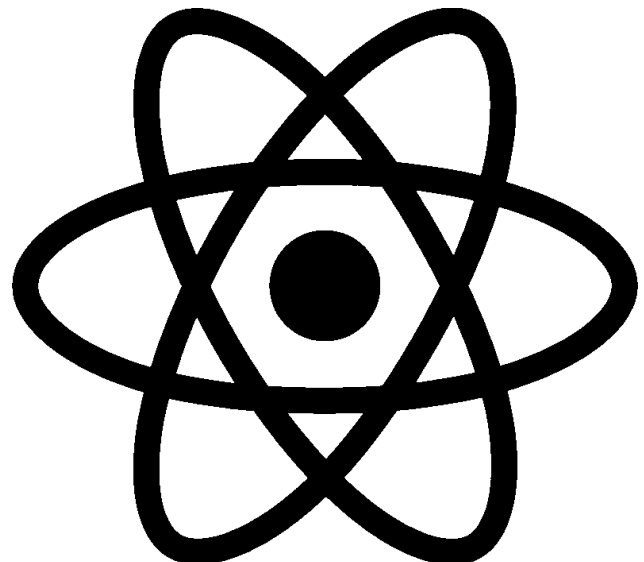
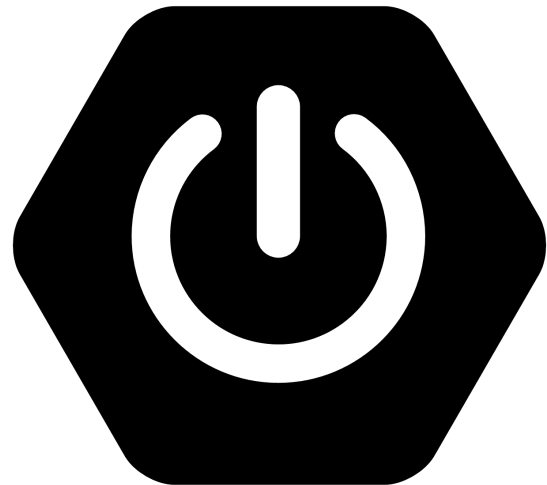
Due 16 November 2025

Web Engineering

Assignment 04

Course Lecturer: Nouman Ali Shah

Section: A



Web Engineering (CSC324) - Assignment 04

Product Inventory, Sales & Purchase Management System

1. System Overview

This project implements a comprehensive Inventory Management System using modern web technologies. The system manages products, customers, suppliers, and handles sales and purchase transactions with robust business rule enforcement.

Technology Stack:

- **Backend:** Spring Boot 3.5.7, Spring Data JPA, MySQL 8.0, Lombok
 - **Frontend:** React 18, Material-UI v5, Axios, React Router
 - **Tools:** IntelliJ IDEA, WebStorm, MySQL Workbench, Postman
-

2. Domain Model and Relationships

The system implements six core entities with well-defined relationships:

Entity	Key Fields	Relationships
Product	id, code (unique), name, unitPrice, currentStock, active	One-to-One with ProductDetail; One-to-Many with Sale and Purchase
ProductDetail	brand, category, description, minStockLevel, taxRate	Extends Product information
Customer	id, name, email (unique), phone, address, blocked	One-to-Many with Sale
Supplier	id, name, email (unique), companyName, address, active	One-to-Many with Purchase
Sale	id, product, customer, date, quantity, totalPrice, status	Transaction record (CONFIRMED/CANCELLED)
Purchase	id, product, supplier, date, quantity, totalCost, status	Transaction record (RECEIVED/CANCELLED)

The database schema follows snake_case naming conventions with proper constraints including unique constraints, foreign key relationships, check constraints for non-negative values, and indexes on frequently queried columns.

3. Business Rules Implementation

3.1 Sales Transaction Rules

Enforced in `SaleService.createSale()` with `@Transactional`:

- **Product Validation:** Product must exist, be active, and have available stock
- **Stock Management:** Available stock must be $\geq$ requested quantity; stock decreases atomically (`currentStock -= quantity`)
- **Customer Validation:** Customer must exist and not be blocked
- **Transaction Integrity:** Sale creation and stock update wrapped in single transaction with automatic rollback on exceptions

```
{
  "timestamp": "2025-11-15T10:30:00",
  "status": 400,
  "error": "Business Rule Violation",
  "message": "Insufficient stock. Available: 5, Requested: 10",
  "path": "/api/sales"
}
```

3.2 Purchase Transaction Rules

Enforced in `PurchaseService.createPurchase()` with `@Transactional`:

- **Supplier Validation:** Supplier must exist and be active
- **Stock Management:** Stock increases atomically (`currentStock += quantity`)
- **Transaction Integrity:** Purchase creation and stock update in single atomic transaction

3.3 Multi-Layer Validation

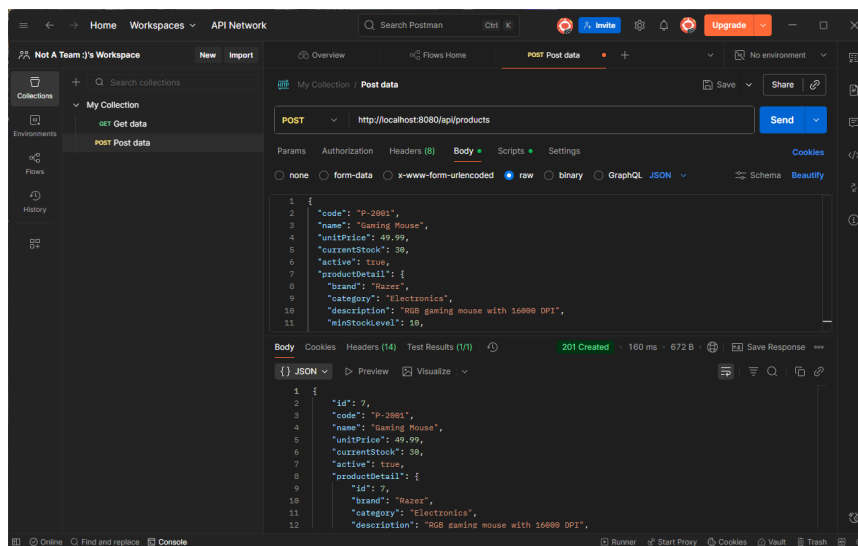
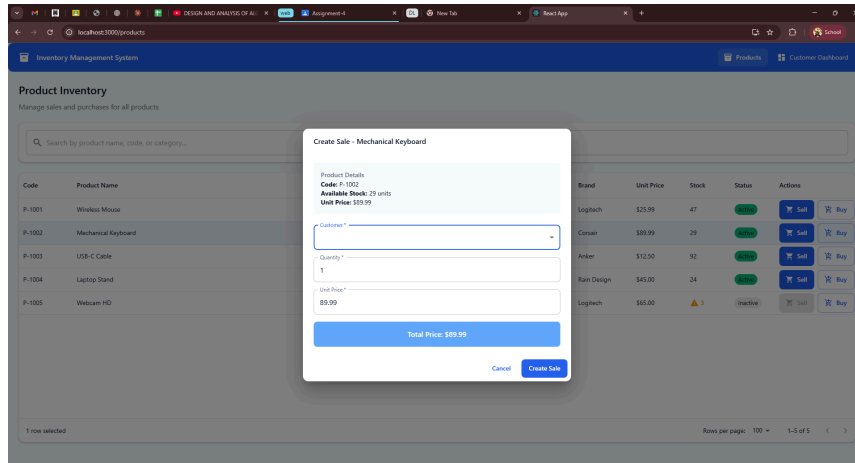
1. **DTO Layer:** Jakarta Bean Validation annotations (`@NotNull`, `@Min`, `@Email`)
2. **Service Layer:** Business logic validation before database operations
3. **Database Layer:** Constraints enforce data integrity at database level

4. Testing and Validation Results

4.1 Successful Operations

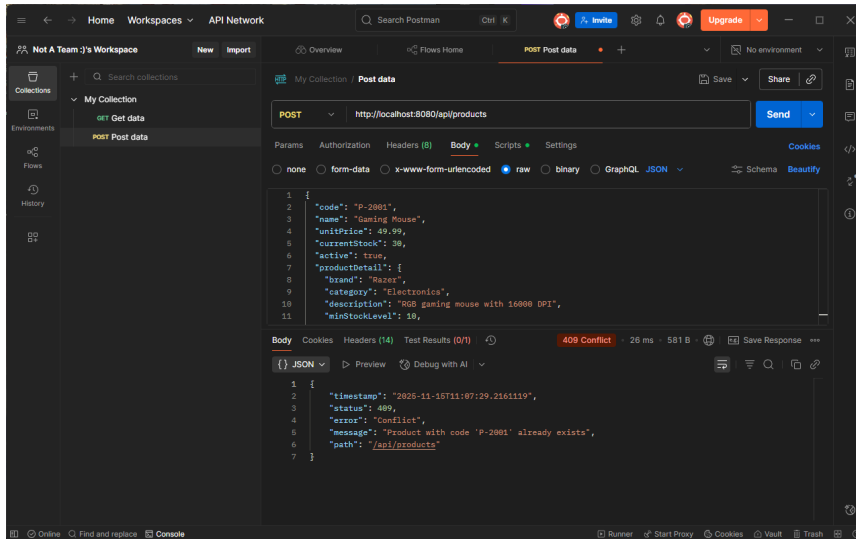
Sale Creation Test: Successfully sold 2 Wireless Mouse units to John Doe

- Response: 201 CREATED
- Stock decreased from 48 to 46 units
- Database verification confirmed CONFIRMED status and correct foreign key relationships



Purchase Creation Test: Successfully purchased 20 Webcams from Tech Distributors Inc

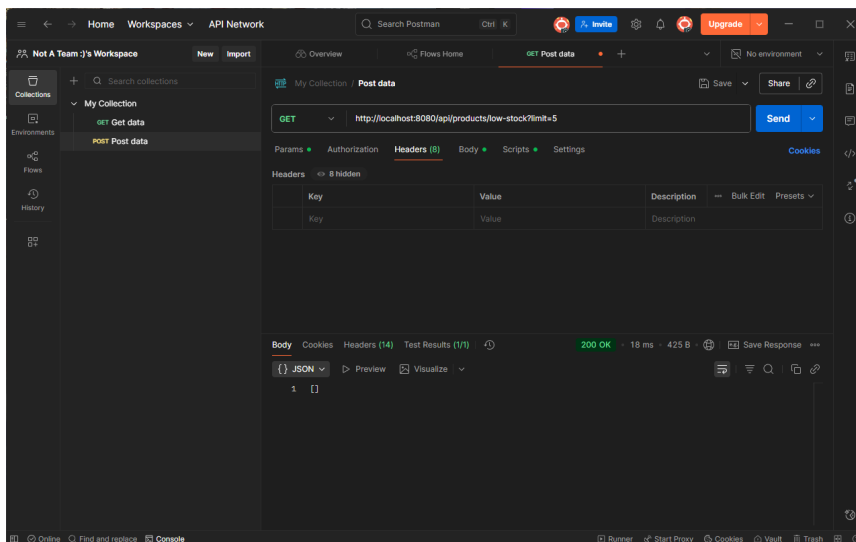
- Response: 201 CREATED
- Stock increased from 3 to 23 units
- Low stock warning automatically removed



4.2 Error Case Validation

Insufficient Stock: Attempted sale of 100 units when only 46 available

- Response: 400 BAD REQUEST with message "Insufficient stock. Available: 46, Requested: 100"
- Transaction rolled back; no data modification occurred



Blocked Customer: Attempted sale with blocked customer (Ali Ahmed, ID: 3)

- Response: 400 BAD REQUEST with message "Customer (ID: 3) is blocked. Cannot create sale."

Inactive Supplier: Attempted purchase from inactive supplier (Global Gadgets, ID: 3)

- Response: 400 BAD REQUEST with message "Supplier (ID: 3) is inactive. Cannot create purchase."
-

5. Bonus Feature: Low Stock Alert

Implementation: Custom JPQL query in `ProductRepository`

```
@Query("SELECT p FROM Product p JOIN ProductDetail pd  
  WHERE p.currentStock < pd.minStockLevel  
  AND p.active = true")  
List<Product> findLowStockProducts();
```

Features:

- API Endpoint: `GET /api/products/low-stock?limit=5`
 - Returns products where `currentStock < minStockLevel`
 - Frontend displays orange warning icons on `ProductList` for low stock items
 - Icons automatically disappear when stock exceeds minimum level
-

6. System Architecture

6.1 Backend (Spring Boot)

- **Layered Design:** Controller → Service → Repository → Entity
- **25+ REST endpoints** across 7 controllers
- **Transaction Management:** `@Transactional` for data consistency
- **DTOs:** Separate request/response objects for API isolation
- **Exception Handling:** `@RestControllerAdvice` for centralized error responses

6.2 Frontend (React)

- **Single-page application** with React Router
- **Key Components:** `ProductList` (DataGrid with search), `SaleForm` (modal with customer selection), `PurchaseForm` (modal with supplier selection), `CustomerDashboard` (sales history with filtering)
- **Material-UI:** Professional, responsive design with consistent UX

- **Real-time Updates:** Automatic refresh on CRUD operations

Inventory Management System

Product Inventory

Manage sales and purchases for all products

Search by product name, code, or category...

Code	Product Name	Category	Brand	Unit Price	Stock	Status	Actions
P-1001	Wireless Mouse	Electronics	Logitech	\$25.99	47	Active	Sell Buy
P-1002	Mechanical Keyboard	Electronics	Corsair	\$89.99	29	Active	Sell Buy
P-1003	USB-C Cable	Accessories	Anker	\$12.50	92	Active	Sell Buy
P-1004	Laptop Stand	Accessories	Rain Design	\$45.00	24	Active	Sell Buy
P-1005	Webcam HD	Electronics	Logitech	\$65.00	12	Inactive	Sell Buy

Rows per page: 100 1-5 of 5

Inventory Management System

Customer Sales Dashboard

View sales history and statistics for any customer

Select Customer

Please select a customer to view their sales history

Inventory Management System

Customer Sales Dashboard

View sales history and statistics for any customer

Select Customer: Jane Smith (jane.smith@example.com)

Total Sales: 3

Confirmed Sales: 2

Total Revenue: \$159.98

Filter by Status: All Statuses

Date	Product Code	Product Name	Quantity	Unit Price	Total Price	Status
Nov 15, 2025, 12:32 AM	P-1001	Wireless Mouse	1	\$25.99	\$25.99	Confirmed
Nov 14, 2025, 11:48 AM	P-1002	Mechanical Keyboard	1	\$89.99	\$89.99	Cancelled
Nov 10, 2025, 11:48 PM	P-1002	Mechanical Keyboard	1	\$25.99	\$25.99	Confirmed

6.3 Data Seeding

Sample data automatically loaded on startup:

- 5 Products (including 1 inactive)
 - 5 Customers (including 1 blocked)
 - 5 Suppliers (including 1 inactive)
 - 5 Initial purchases (set stock levels)
 - 6 Sample sales (including 1 cancelled)
-

7. Conclusion

This project successfully demonstrates a production-ready full-stack web application with complete CRUD operations, robust business rule enforcement (8+ validation rules), atomic inventory management, professional Material-UI interface, comprehensive error handling, and low stock alert system. The implementation showcases mastery of Spring Boot, Spring Data JPA, RESTful API design, React state management, transaction management, and full-stack integration.